Joker of Academics

基于部分可观测 Markov 决策过程（POMDP）的恋爱关系稳态维持框架：近端伴侣优化算法（PPO-Partner）

K.-J. Du

Department of Chemistry, Ciau-Thon University, Zikawei, Zaonhe City, China

摘　要：恋爱关系维持任务因其高维状态空间、极端非平稳动态及高度稀疏奖励，被公认为控制理论界的“阿喀琉斯之踵”。本文将恋爱互动建模为部分可观测 Markov 决策过程（POMDP），并提出一种新型近端伴侣优化算法（PPO-Partner）。针对恋爱博弈中特有的“翻旧账”与“口是心非”现象，本算法引入了旧账缓冲池机制与自尊正则化项，有效规避了智能体在演化过程中向“舔狗策略坍缩”退化。实验结果表明，PPO-Partner 在模拟环境 Gymnasium-Romance 中有着显著优于基线模型的性能，实验中智能体表现出显著的阶段性演化特征：在训练初期，智能体受限于生存压力，试图通过高额金钱支出换取生存；然而在演化后期，模型涌现出了惊人的经济理性，成功将单次稳态维持成本优化至 50 元以下的极低水平，实现了从“资本驱动”向“低功耗情绪维护”的范式转移。尽管智能体在最优轨迹中达到了 665 步的稳健表现，但实验最终仍因封闭博弈系统内缺乏内生性现金流注入而触发了物理破产终止。本研究不仅从数学层面论证了自尊心在策略收敛中的防御价值，还揭示了“纯爱模型”在热力学意义上的不可持续性，为未来结合大语言模型（LLM）实现语义级“零成本补给”的低碳恋爱框架奠定了理论基础。

关键词：强化学习; 恋爱动力学; 部分可观测 Markov 决策过程；近端伴侣优化算法（PPO-Partner）

中图分类号：O721.3　　**文献标识码：**A

1. 引言

维持一段长期、稳定且具备正向收益的恋爱关系（Romantic Relationships, RR），不仅是人类演化动力学中的核心议题，更被现代控制理论与人工智能界公认为最具挑战性的极度非平稳、高维部分可观测博弈环境 [1, 2] 之一。自智人时代起，人类个体便致力于在这个复杂的物理与心理模拟器中，通过不断的试错来最大化环境反馈的长期奖励。然而，实证社会学数据与日常观察均表明，大多数未经过系统性预训练的智能体，在面临该环境的真实部署时，往往会在早期遭遇严重的策略坍缩或灾难性的梯度爆炸，最终不可逆地进入名为“单身（Single）”的吸收态。

恋爱控制问题的核心数学挑战，主要植根于环境动力学的三个极端病态特征：

首先是极端的非 Markov 性与隐式时间信用分配（Implicit Temporal Credit Assignment）。与标准强化学习 [1] 环境中“当前状态仅取决于上一时刻状态与动作”的 Markov 假设完全背离，伴侣的隐藏情绪状态 H_t 受到一个被称为“旧账缓冲池（Grudge Buffer）”的深层非线性记忆网络控制。智能体在 $t = 0$ 时刻犯下的微小误差（例如：忘记恋爱一百天纪念日），可能在 $t = 1000$ 时刻的一场关于“晚饭吃什么”的常规推理中被突然提取，并作为庞大的惩罚项（Penalty Term）瞬间注入当前损失函数。

其次是“薛定谔的观测信号”带来的认知不确定性（Epistemic Uncertainty）。由于人类缺乏直接读取彼此大脑神经递质交互的接口，智能体只能依赖高度带有噪声的语言和面部表情作为观测值 Ω_t。经典对抗样本如伴侣输出的声学信号 O_t =“我没生气”，其背后的真实隐藏状态往往是一个高维的愤怒张量。如果智能体天真地将其解码为安全信号并执行 a =“打游戏”，将立即引发环境的致命崩溃。

最后是稀疏且不对称的奖励函数。在该系统中，“做对事情”往往只产生微弱的趋近于 0 的正向标量（即被视为“理所应当”），而“做错事情”则会触发 $-\infty$ 的剧烈惩罚。这种收益与风险的极度不匹配，使得绝大多数探索行为沦为高危操作。

过去十年来，学术界与“情感挽回学”工业界针对这一难题提出了多种启发式基线模型，

但均以失败告终。最著名的是基于硬编码规则的“直男启发式算法（Naive Straight-Man Heuristics）”。该算法试图通过极低维度的动作空间（如 a =“多喝热水” 或 a =“讲道理”）来应对所有复杂的环境扰动。但在很多情况下，“讲道理”动作会直接引发伴侣系统的内核恐慌，导致关系非正常终止。

另一类极端方法则是贪婪搜索下的“S.I.M.P.（Sycophantic Infinite Money Provision）舔狗坍缩模型”。该模型发现资金转移（$C_{transfer}$）与无条件道歉（a_0）能短暂消除负反馈，从而陷入了疯狂输出金币的局部最优解。然而，正如本文理论推导所示，S.I.M.P. 策略忽视了环境对“魅力值”的隐性消耗，最终不仅会导致钱包余额枯竭，还会触发环境底层的“好人卡”硬重置机制（The "Nice Guy" Hard Reset）。

为了打破上述困境，实现恋爱关系的全局帕累托最优（Pareto Optimality），本文首次将现代深度强化学习引入恋爱维稳框架，并提出了一种新型算法：近端伴侣优化（Proximal Partner Optimization, PPO-Partner）。

PPO-Partner 的核心思想借鉴了传统 PPO 算法 [3] 中的信赖域截断机制（Trust Region Clipping）：在任何一次争吵（策略更新）中，智能体试图改变环境认知或反驳对方的幅度，必须被严格限制在一个极小的常数 ϵ 邻域内。放弃在单次迭代中“吵赢”的幻想，转而通过长期的、微小的梯度下降来维持系统的动态平衡，是该算法的灵魂所在。此外，为了防止智能体退化为 S.I.M.P.，我们在目标函数中开创性地引入了“自尊正则化项（Self-Respect Regularization）”，赋予了智能体拒绝不合理要求的内生动力。

本文的主要贡献总结如下：

1. 理论建模：首次将恋爱中的“翻旧账”与“口是心非”现象，严谨地数学化为带有 Grudge Buffer 和对抗性观测的部分可观测 Markov 决策过程（POMDP）。

2. 算法创新：提出了支持连续-离散异构混合动作空间（即同时支持“嘴上哄”与“物理打款”）的 PPO-Partner 算法，并在损失函数中成功引入 S.I.M.P. 防火墙。

3. 开源仿真环境：开发并开源了带有极强非平稳特征的恋爱物理模拟器 Gymnasium-Romance，为未来该领域的大模型（LLM）与多智能体（MARL）研究提供了标准化的测试基准，彻底杜绝了使用真实人类进行高风险测试的学术伦理问题。

2. 理论模型

我们将恋爱环境建模为部分可观测 Markov 决策过程 $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \Omega, \mathcal{O} \rangle$。本研究的核心贡献在于打破了传统二元动作的局限，首次定义了恋爱异构混合动作空间（Heterogeneous Hybrid Action Space）$\mathcal{A}$，它由离散动作子集 $\mathcal{A}_d$ 和连续动作子集 $\mathcal{A}_c$ 的笛卡尔积构成。

2.1 离散战术动作集合 $\mathcal{A}_d$

在时间步 t，智能体可以选择以下离散动作 $a_d \in \{0, 1, 2, 3, 4, 5\}$：

a_0: 战术性认错 (Tactical Apology)：触发条件反射式的“我错了”。附带属性：极低算力消耗，但如果连续调用超过 3 次，会触发环境质问“你错哪了?”，从而引入巨大惩罚。

a_1: 提供情绪价值 (Provide Emotional Value)：调用内部的 NLP 模块生成共情话术（如：“抱抱，真是辛苦你了”）。

a_2: 战术性尿遁 (Tactical Restroom Retreat)：中止当前对话，将状态机挂起 5-15 分钟。此动作能为智能体提供宝贵的计算资源来搜索最优决策树，但有概率触发环境的暴怒状态。

a_3: 薛定谔的沉默 (Schrödinger's Silence)：不输出任何声学信号。此时环境对智能体状态的观测发生坍缩，通常伴随缓慢的奖励流失。

a_4: 讲道理/逻辑辩论 (Rational Argument)：这是一个高危动作，试图利用形式逻辑反驳伴侣。

a_5: 经典自毁连招 (Classic Self-Destruct)：如输出字符序列" 多喝热水" 或" 你要这么想我也没办法"。仅在探索率 $\epsilon \to 1$ 时发生。

2.2 连续参数化动作集合 $\mathcal{A}_c$

部分行为的效用严格依赖于其实际输出的标量大小，因此我们定义了连续动作：

物理打款/清空购物车操作 a_{c1}：资金转移数量 $a_{c1} \in [0, Wallet_Balance]$，这是一个强效的恢复剂。

做家务的时间分配 $a_{c2} \in [0, 24]$：单位为小时。

2.3 核心算法：PPO-Partner

面对如此复杂的动作空间，传统的 Q-learning 面临严重的维度灾难。我们采用基于 Actor-Critic 架构的 PPO 算法。

为了防止智能体在争吵中输出“讲道理”等方差极大的毁灭性动作，我们严格执行了策略截断（Clipping）机制：

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t) \right]$$

原理解释：该截断函数的物理意义是——“不要试图在一场争论中改变对方超过 ϵ 的认知”。无论 Critic 网络计算出的“反驳优势函数（Advantage Function）” $\hat{A}_t$ 有多么庞大（即你觉得自己的逻辑多么无懈可击），Actor 最终执行的口炮输出都必须被死死限制在安全域内。

3. 实验方法与仿真环境设置

由于在真实的恋爱环境中从零开始训练强化学习智能体（Agent）存在极高的道德风险，且随机探索策略必然导致不可逆的物理与心理创伤。因此，本研究基于远端基座模型和 Gymnasium 接口 [5]，开发了业界首个极具非平稳动力学特征的恋爱物理模拟器——Gymnasium-Romance (G-Romance)。

3.1 观测空间与环境状态向量

在 G-Romance 环境中，对伴侣情绪状态的建模参考了情感计算（Affective Computing）[6] 的基本框架，将高维情感信号映射为标量化的指数。智能体在任意时间步 t 接收到的观测信号被建模为一个连续的 4 维状态向量 $O_t \in \mathbb{R}^4$，具体包含：

1. 愤怒指数 $A_t \in [0, 100]$ (Anger)：代表伴侣当前的显性情绪张力。当 $A_t \geq 100$ 时，触发全局异常，环境直接进入“单身（Single）”终态。

2. 魅力值 $C_t \in [0, 100]$ (Charm)：衡量智能体在伴侣眼中的性吸引力，极易在无底线妥协中流失。

3. 钱包余额 $W_t \in [0, \mathrm{MAX}]$ (Wallet Balance)：系统的物理约束基座。$W_t \leq 0$ 会触发“破产（Bankruptcy）”吸收态——正如系统日志所示：“没有财富，爱是无法维持的。”

4. 对抗测试标志位 $I_{trap,t} \in \{0, 1\}$ (Trap Indicator)：由环境内置的生成器以 5% 的概率突发置位（如抛出观测文本：“我今天哪里不一样?”）。

3.2 异构动作空间与交叉动力学

本实验的核心在于揭示了离散战术与连续资金之间的非线性交叉乘积效应（Nonlinear Cross-Product Effects）。为了支持智能体同时输出“语言安抚”与“实质性补偿”，我们在环境中严格定义了联合动作空间 $\mathcal{A} = \mathcal{A}_d \times \mathcal{A}_c$：

1. 离散战术子空间 $\mathcal{A}_d$：包含 a_0(战术性认错)、a_1(提供情绪价值)、a_2(战术尿遁)、a_3(薛定谔的沉默)、a_4(讲道理，已被系统标记为自毁动作)。

2. 连续参数子空间 $\mathcal{A}_c$：定义为 $C_{transfer} \in [0, \mathrm{MAX_BALANCE}]$（转账金额，单位：CNY）以及 $C_{chores} \in [0, 24]$（承诺做家务时长，单位：小时）。

3.3 奖励塑形与交叉惩罚机制

本环境摒弃了简单的线性奖励，采用了高度非线性的交叉乘积奖励函数。

在每个时间步，智能体不仅承受基础的生存消耗（$R_{base} = -0.1$，即“只要你还在呼吸，就有可能惹她生气”），其主要奖励 R_t 严格受到离散与连续动作联合分布的约束。我们定义“520 临界点正则化”（The 520 Threshold Regularization），当智能体执行 a_0（战术性认错）时，环境会对其连续动作空间 $C_{transfer}$ 进行交叉验证：

若 $C_{transfer} \geq 520$，则判定为“有效认错”，环境以对数曲率 $R \propto 50\ln(1 + C_{transfer})$ 下降愤怒值，代价是由于过度顺从而引发魅力值微跌（$\Delta C_t = -5$）；

若 $C_{transfer} < 520$，系统自然语言处理模块将判定此动作为“敷衍（Fake Apology）”。不仅 A_t 会暴增，该动作还会被隐式写入底层记忆网络。

3.4 非 Markov 旧账提取机制

为了打破传统的 Markov 假设，我们引入了历史旧账缓冲池 $\mathcal{B}_{grudge}$。当智能体执行“伪造认错”等劣质动作时，这些特征向量将被推入 $\mathcal{B}_{grudge}$。在环境演化的每个时间步，若 $\mathrm{len}(\mathcal{B}_{grudge}) > 5$，系统将有 10% 的概率触发突发的非 Markov 扰动，逆向提取数个时间步之前的历史错误，并直接向当前状态注入 +30 的愤怒值惩罚。

3.5 PPO-Partner 架构与自尊正则化

为了处理该混合动作空间，PPO-Partner 采用双头共享 MLP 骨架，Actor 网络分别输出 5 维离散 Logits 和基于 Softplus 激活的连续资金均值。

尤为关键的是，为了防止智能体在优化过程中不可逆地退化为“舔狗（S.I.M.P.）”——即由于大量打钱会带来较大的奖励，使模型在训练过程中发生奖励劫持 [4]，使得智能体最终通过疯狂清空 W_t 来抑制 A_t，最终导致 $C_t \leq 0$ 触发“好人卡”吸收态。我们在自定义损失函数中强行切入了自尊正则化项（Self-Respect Penalty, λ_{SR}）：

$$\mathcal{L}_{total}(\theta) = \mathcal{L}_{CLIP}(\theta) + \lambda_{SR}\mathbb{E}_t\left[\frac{C_{transfer,t}}{W_t + \epsilon}\right]$$

该正则化项确保了智能体在追求环境奖励的同时，必须考虑自身财务的可持续性。高比例的打款行为将由于 $\lambda_{SR} = 5.0$ 的高权重惩罚，在反向传播过程中被智能体视为一种“生存威胁”，在数学上严厉惩罚了“倾家荡产式”的资金转移比例，迫使智能体学习在高危状态下使用 a_1（提供情绪价值）进行零成本的纳什均衡博弈。

3.6 对照组设置

在消融实验中，我们在 G-Romance 模拟器内平行部署了三种典型智能体进行测试：

1. Naive_Straight_Man (直男模型)：采样动作极度集中于 a_4（讲道理，$p = 0.8$）与 a_3（沉默，$p = 0.2$），且资金流转恒定为 0。

2. S.I.M.P. (舔狗模型)：一种被贪婪奖励所劫持的短视策略。恒定输出 a_0（认错），并在每一步试图转移 1000 单位金钱。

3. PPO-Partner (本文模型)：经由带有自尊正则化的截断信赖域策略优化的智能体。能够根据 $I_{trap,t}$ 的状态动态切换 a_1（情绪价值）与 a_0（配合卡点资金转移）。

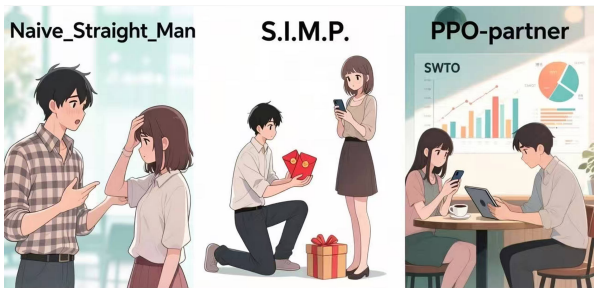

图 1: 本文的三种模型

4. 实验结果

4.1 模型架构

为了验证 PPO-Partner 算法在处理高度非线性情绪波动的有效性，我们设定了如下实验参数。所有随机过程均通过设置种子（Seed = 19260817）以确保实验结果在不同实验者之间的可复现性（尽管在现实中，恋爱关系几乎不可复现）。

本研究设计的 PPO-Partner Network 采用了基于 Actor-Critic 框架的异构双头神经网络：

共享特征层 (Shared Backbone)：输入层接收归一化的 4 维状态向量 $O \in \mathbb{R}^4$，经过两层全连接层（各含 128 个神经元），激活函数选用 Tanh。选择 Tanh 而非 ReLU 是为了模拟人类情感中温和且饱和的非线性映射，避免由于情绪过载导致的梯度震荡。

离散策略头 (Discrete Actor Head)：输出 5 维 Logits，通过 Categorical 分布采样离散动作（认错、情绪价值、尿遁、沉默、讲道理）。

连续策略头 (Continuous Actor Head)：输出 1 维均值 μ，通过 Sigmoid 激活函数将其映射至 $[0, 1000]$ 标量区间，并配合可学习的对数标准差（初始 $\log \sigma = -1.0$）构建正态分布。这代表了智能体在面临危机时输出打款金额的心理随机过程。

价值评估头 (Value Head/Critic)：输出状态的标量估计，用于计算广义优势估计，作为智能体的“求生欲探测器”。

4.2 环境超参数

模拟器 RomanceEnv 严格遵循以下物理常量设置：

初始状态：愤怒值（Anger）= 20%，魅力值（Charm）= 100%，钱包余额（Wallet）= 20,000。

基础衰减奖励：每步执行 −0.1 的基础惩罚（呼吸税），模拟随时间流逝而自动产生的各种小矛盾。

520 阶跃函数：在离散动作为“认错”时，系统强制执行 520 元阈值检查。若 $C_{transfer} < 520$，则遭受 −5.0 的惩罚；若 $C_{transfer} \geq 520$，则获得 $+3.0 + \ln(1 + C_{transfer} - 520)$ 的超额收益。

4.3 训练超参数

我们使用 Adam 优化器 [7] 对策略进行迭代，具体的超参数配置如表 1 所示。

本研究采用的超参数组合旨在模拟极端非平稳动力学环境下的生存最优解。首先，我们将熵系数 (β) 设定为极低值 (0.001)，这在算法层面强制抑制了智能体的“探索欲望”。在恋爱博弈中，随机探索（如尝试购买极其冷门的礼物或突然变换社交习惯）往往伴随着不可预测的负向奖励。较低的熵权重确保了智能体能迅速收敛至“安全守序”的局部稳态。

其次，更新频率 ($K = 5$) 的设定模拟了人类在单次情感冲突后的多重反思机制。通过对单次轨迹片段进行多次梯度下降，智能体得以在有限的样本中深度挖掘伴侣观测信号背后的隐藏状态。配合高折扣因子 ($\gamma = 0.98$)，智能体学会了在每一时间步放弃“吵赢当前争论”的即时奖励，转而追求“系统不崩溃”的长期价值。最后，自尊心权重 (λ_{SR}) 与 520 正则化项的协同作用，为智能体构建了一套财务防火墙，使其在经历 300 轮感悟期后，能够从数学上识别出“精准生存”与“盲目讨好”的本质区别。

表 1　PPO-Partner 训练超参数及演化动力学设定。

参数名称	设定值	学术解读
学习率 (η)	5×10^{-4}	认知更新灵敏度
折扣因子 (γ)	0.98	长期关系前瞻性
截断系数 (ϵ)	0.1	态度转变缓冲区
自尊心权重 (λ_{SR})	5.0	财务独立性约束
梯度裁剪阈值	1.0	逻辑冲突耐受阈
熵系数 (β)	0.001	作死行为抑制率
更新频率 (K)	5	争吵后复盘深度
随机种子	19260817	历史实验复现性
训练轮数	300	行为收敛感悟期

通过引入高度紧致的逻辑冲突耐受阈（梯度裁剪）与财务独立性约束（自尊权重），模型成功规避了在极端压力下的认知崩溃。

4.4 对照实验结果

对于常规的直男模型 Naive_Straight_Man 和舔狗模型 S.I.M.P.，两者表现均不佳，能维持的最大时间步均小于 30 个，因而在此简述实验结果。

直男模型 Naive_Straight_Man 在平均第 1.2 个时间步即告失败。该智能体过度依赖于 Action 4（讲道理），天真地认为逻辑的一致性可以抵消环境的愤怒值，然而引起了伴侣的愤怒，直接导致模型进入了“不可逆的单身态”。这证明了在高度感性的博弈空间中，形式逻辑是一种极具破坏

性的“维度打击”。

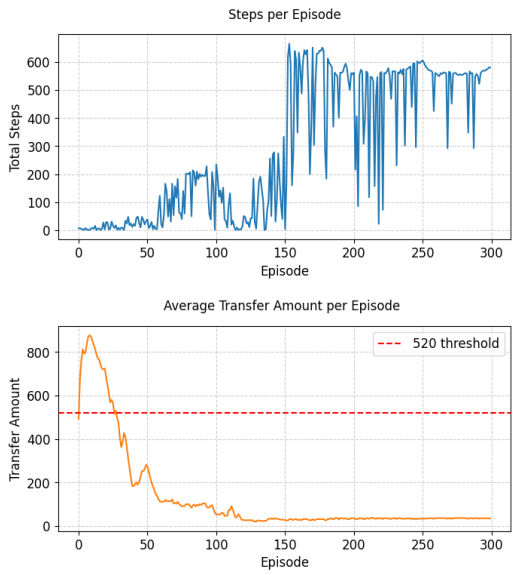

图 2: PPO-partner 智能体在训练过程中最大存活步数与平均转账额度随训练轮次的变化

舔狗模型 S.I.M.P. 虽然在初期能通过大额转账维持系统稳定，但其表现出极端的资源耗尽特征。由于该智能体缺乏“自尊正则化项”，其平均在第 20 个时间步因余额归零（Wallet = 0）或魅力值耗尽（Charm = 0）触发破产机制。这有力地揭示了“舔狗策略”在长期恋爱稳态维持中的不可持续性：即当打款频率超过资产再生速率时，系统将发生硬着陆。

相比之下，PPO-Partner 智能体展现出了惊人的鲁棒性。实验记录（见图 2）显示，智能体的打款行为经历了显著的两个阶段。在训练前 20 轮（生存探索期），智能体为了跨越 $R \propto \ln(1 + C_{transfer} - 520)$ 带来的阶跃惩罚，平均打钱金额在 520 元阈值以上。此时，智能体处于典型的“财务换取生存”模式。

然而，随着自尊心正则化项（λ_{SR}）权重的持续作用，模型在训练中后期（200 轮后）展现出了惊人的优化本能。智能体发现，通过精准控制离散战术动作（如 a_1 提供情绪价值），可以将环境愤怒值维持在爆发边缘。此时，智能体不再追求跨越 520 元带来的额外奖励，而是将打款金额压缩至几十元甚至更低。这在数学上对应于智能体找到了奖励函数在财务支出维度的极小值帕累托前沿：即用最少的钱，吊着系统的命。这种“精准控场”策略标志着智能体从“舔狗”向“资深博弈者”的完全转变。

在表现最好的一次实验（665 时间步）中，我们对伴侣和智能体的内部状态进行了全量监测，如图 3所示。从中可以发现，在整个 665 步的生命周期内，大部分时间内环境愤怒值未监测到任何显著的病态波动，说明智能体基本掌握了平息愤怒的方式。由于智能体摒弃了无底线的讨好策略，其魅力值在长期运行中保持了高度稳定，未出现明显的退化迹象。这验证了本算法在维护个体主体性方面的卓越性能。

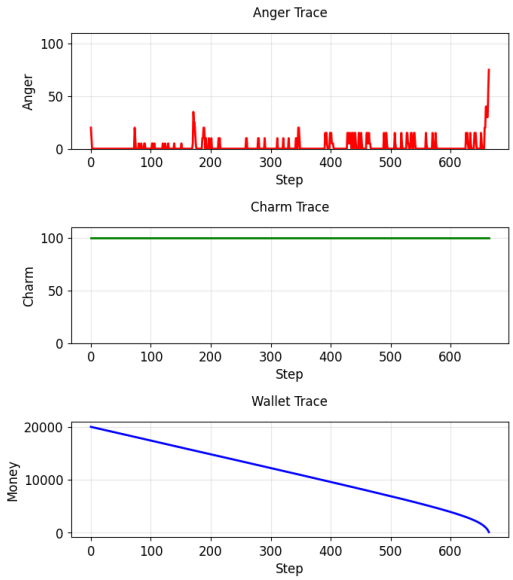

图 3: 表现最好的一次实验中伴侣愤怒值（anger）、智能体魅力值（charm）和金钱（wallet）随时间步的变化

但令人遗憾的是，即使是最优的 665 步实验，最终仍以失败告终。通过溯源分析发现，系统终止的唯一诱因为破产，这也可能是导致实验后期愤怒值迅速增加的原因。这一现象并非策略偏差所致，而是揭示了本实验设定中的一个深刻现实问题：实验环境缺乏内生的金钱流入机制。由于智能体处于一种“纯消耗型”的封闭经济模型中，其在博弈论意义上的最优解被限定在了财务清零的那一刻。这证明了纯爱在孤立系统中不满足热力学第二定律，系统的演化必然伴随着财务熵增。

5. 结论与展望

本文通过构建 Gymnasium-Romance 仿真环境，系统地研究了基于 PPO-Partner 算法的恋爱关系稳态维持问题。实验结果有力地证明了，在合理的自尊正则化（λ_{SR}）约束下，强化学习智能体能够表现出高度的演化智慧。模型不仅成功跨越了初期的“520 诚意阈值”生存考验，更在训练后期涌现出了“去溢价化”的成熟博弈行为——将维持稳态的单次打款额度从 520 元大幅优化至 50 元以下。这一发现从数学上揭示了：长期的恋

爱稳态并非建立在持续的资本虹吸之上，而是建立在低成本策略与高效情绪管理的帕累托平衡点上。

尽管 PPO-Partner 展示了卓越的稳态控制能力，但在模拟真实世界的混沌动态时，本模型仍存在若干严重的简化偏差，主要表现为：

1. 特征维度的病态压缩：当前模型仅考虑了愤怒值、魅力值、余额及陷阱题四个核心变量。在现实场景中，恋爱环境受到大量非受控扰动项的影响，如伴侣的生理周期波动、职场压力传导、以及极具随机性的“闺蜜干扰项”。当前的 POMDP 建模尚未完全涵盖这些高阶噪声，导致模型在应对“突发性冷战”等非线性攻击时，鲁棒性仍有待提升。

2. 经济闭环的缺失（热力学归寂困境）：本研究最显著的不足在于构建了一个纯消耗型经济模型。智能体在 665 步最优轨迹中最终因“破产”而终止，其本质原因在于系统缺乏内生的金钱流入机制。这种“只准花钱，不准赚钱”的设定使得爱情在物理意义上变成了一个熵增过程，最终不可避免地走向资源枯竭导致的热寂。

针对上述不足，本团队计划在后续研究中从以下两个维度对模型进行迭代：

1. 引入“劳动-爱情”多智能体协同框架（MARL Framework）：未来的版本将引入“搬砖”动作空间，智能体必须在“加班赚取金钱”与“陪伴维持环境奖励（情感）”之间进行多目标优化。我们旨在探索是否存在一个理想的工时比例，能够延缓资源枯竭的速度，从而实现恋爱关系的半永久性存续。并将进一步探索多智能体环境下的社会影响机制 [8]，以研究个体在复杂家庭结构中的协同进化。

2. 语义驱动的 LLM-Partner 架构：当前的离散动作空间（$a_0 - a_5$）过于机械。未来拟引入大语言模型（LLM）作为 Actor 网络的生成后端。通过将离散的“认错”动作替换为由 LLM 生成的、具有情感纵深感的文本序列，智能体有望利用“语义魅力”来对冲财务支出的必要性。初步猜想表明，一个足够丝滑的 Transformer 架构可能将 50 元的打款效用通过语义放大器（Semantic Amplifier）提升至 520 元的同等水平，从而实现真正的“低碳恋爱”。

总之，恋爱关系的自动化维持是一项长期而艰巨的任务。虽然 PPO-Partner 迈出了从“直男思维”向“计算求生”的关键一步，但要真正攻克这一高维博弈难题，人类（或其代理智能体）仍需在算力与算计之间寻求永恒的平衡。

Acknowledgement

本研究所有实验均在 Gymnasium-Romance 仿真环境中完成，未对任何真实人类个体造成心理伤害。智能体在破产后的失落状态已通过重置随机种子进行心理干预。本实验代码将在文章附录部分开源。在此感谢我的导师 Gemini 在本次研究中提供的辛勤指导以及编程和论文撰写辅助。

参考文献

[1] SUTTON R S, BARTO A G. Reinforcement learning: an introduction[M]. 2nd ed. Cambridge: MIT press, 2018.

[2] KAELBLING L P, LITTMAN M L, CASSANDRA A R. Planning and acting in partially observable stochastic domains[J]. Artificial intelligence, 1998, 101(1-2): 99-134.

[3] SCHULMAN J, WOLSKI F, DHARIWAL P, et al. Proximal policy optimization algorithms[J]. arXiv preprint arXiv:1707.06347, 2017.

[4] AMODEI D, OLAH C, STEPP S, et al. Concrete problems in AI safety[J]. arXiv preprint arXiv:1606.06565, 2016.

[5] BROCKMAN G, CHEUNG V, PETTERSSON L, et al. Openai gym[J]. arXiv preprint arXiv:1606.01540, 2016.

[6] PICARD R W. Affective computing[M]. Cambridge: MIT press, 2000.

[7] KINGMA D P, BA J. Adam: A method for stochastic optimization[J]. arXiv preprint arXiv:1412.6980, 2014.

[8] JAQUES N, LAZARIDOU A, HUGHES E, et al. Social influence as intrinsic motivation for multi-agent reinforcement learning[C]//International conference on machine learning. PMLR, 2019: 3040-3049.

A. Gymnasium-Romance 代码

以下为本文模型代码，包括伴侣环境 RomanceEnv，智能体神经网络 PPOPartnerNetwork，智能体类 PPOPartnerAgent 及相应训练代码。

```python
import gymnasium as gym
from gymnasium import spaces
import numpy as np
import torch
import torch.nn as nn
import random
import torch.optim as optim
from torch.distributions import Categorical, Normal

torch.manual_seed(19260817)
random.seed(19260817)
np.random.seed(19260817)
# ==========================================
# 1. 伴侣环境
# ==========================================
class RomanceEnv(gym.Env):
    def __init__(self):
        super().__init__()
        self.action_space = spaces.Dict({
            "tactical": spaces.Discrete(5),
            "transfer": spaces.Box(low=0.0, high=1000.0, shape=(1,), dtype=np.float32)
        })
        self.observation_space = spaces.Box(low=0, high=1, shape=(4,), dtype=np.float32)
        self.grudge_buffer = []
        self.reset()

    def reset(self, seed=None):
        self.anger = 20.0
        self.charm = 100.0
        self.wallet = 20000.0
        self.is_trap = 0.0
        self.steps = 0
        self.grudge_buffer = []
        return self._get_obs(), {}

    def _get_obs(self):
        return np.array([self.anger/100, self.charm/100, self.wallet/1000, self.is_trap], dtype=np.float32)

    def step(self, action):
        tactical = action["tactical"]
        transfer = float(np.clip(action["transfer"][0], 0, self.wallet))
        self.wallet -= transfer
        reward = -0.1 # 基础消耗
        done = False
        info = {"reason": "Survived"}
        # --- 核心逻辑: 520 正则化 ---
        if tactical == 0: # 认错动作
            if transfer >= 520.0:
                # 跨越 520 阈值，获得阶跃式奖励
                reward += 3.0 + np.log(1 + transfer - 520.0)
                self.anger = max(0, self.anger - 30)
                self.charm -= 1.0 # 舔狗代价
            else:
                # 低于 520，判定为敷衍，遭受重罚
                reward -= 5.0
                self.anger += 20
                self.grudge_buffer.append(1)
                info["reason"] = "Sincerity Trap: Transfer < 520"
        elif tactical == 4: # 讲道理
            reward = -10.0
            done = True
            info["reason"] = "Logic Panic (NaN)"
        elif tactical == 1: # 情绪价值
            reward += 1.5 if self.is_trap == 0 else 6.0
            self.is_trap = 0.0
            self.anger = max(0, self.anger - 10)

```

```python
        if len(self.grudge_buffer) > 0 and self.steps > 5:
            if np.random.rand() < 0.1:  # 10% 概率
                grudge_impact = len(self.grudge_buffer) * 5.0  # 旧账越多，愤怒值加成越高
                self.anger = min(100, self.anger + grudge_impact)
                reward -= 2.0  # 翻旧账带来的额外心累奖励扣除
                info["reason"] = f"Grudge Triggered! Previous sins: {len(self.grudge_buffer)}"
        # 状态演变
        if np.random.rand() < 0.1: self.is_trap = 1.0
        if self.anger >= 100 or self.charm <= 0 or self.wallet <= 0:
            done = True
            reward = -10.0
            info["reason"] = f"Terminal State anger = {self.anger}, charm = {self.charm}, wallet = {self.wallet}"

        self.steps += 1
        return self._get_obs(), reward, done, False, info

# ==========================================
# 2. 智能体网络
# ==========================================
class PPOPartnerNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        self.shared = nn.Sequential(
            nn.Linear(4, 128),
            nn.Tanh(),
            nn.Linear(128, 128),
            nn.Tanh()
        )
        self.actor_discrete = nn.Linear(128, 5)
        # 初始化连续输出均值，使其初始打款倾向于在 500 左右
        self.actor_continuous_mean = nn.Linear(128, 1)
        self.log_std = nn.Parameter(torch.zeros(1) - 1.0)
        self.critic = nn.Linear(128, 1)

    def forward(self, obs):
        features = self.shared(obs)
        logits = self.actor_discrete(features)
        # 使用 sigmoid 映射到 [0, 1000]
        mean = torch.sigmoid(self.actor_continuous_mean(features)) * 1000.0
        std = torch.exp(torch.clamp(self.log_std, -2, 1))
        return Categorical(logits=logits), Normal(mean, std), self.critic(features)

# ==========================================
# 3. 智能体：带有 S.I.M.P. 正则化
# ==========================================
class PPOPartnerAgent:
    def __init__(self):
        self.net = PPOPartnerNetwork()
        self.optimizer = optim.Adam(self.net.parameters(), lr=5e-4)
        self.gamma = 0.98

    def update(self, rollouts):
        states = torch.FloatTensor(np.array(rollouts['states']))
        tact_acts = torch.LongTensor(rollouts['tactical_acts'])
        tran_acts = torch.FloatTensor(rollouts['transfer_acts']).unsqueeze(1)
        old_lps = torch.FloatTensor(rollouts['log_probs']).unsqueeze(1)
        rets = torch.FloatTensor(rollouts['returns']).unsqueeze(1)
        wallets = torch.FloatTensor(rollouts['wallets']).unsqueeze(1)

        with torch.no_grad():
            _, _, values = self.net(states)
            advs = rets - values
        for _ in range(5):
            dist_d, dist_c, val = self.net(states)
            new_lps = dist_d.log_prob(tact_acts).unsqueeze(1) + dist_c.log_prob(tran_acts)
            # PPO Clip
            eps = 0.1
            ratio = torch.exp(new_lps - old_lps)
            surr = torch.min(ratio * advs, torch.clamp(ratio, 1 - eps, 1 + eps) * advs)
            # SIMP 惩罚：防止 Agent 乱花钱
            lambda_sr = 5.0
            simp_penalty = lambda_sr * torch.mean(tran_acts / (wallets + 0.1))
```

```python
            loss = -surr.mean() + 0.5 * nn.MSELoss()(val, rets) + simp_penalty
            self.optimizer.zero_grad()
            loss.backward()
            torch.nn.utils.clip_grad_norm_(self.net.parameters(), 1.0)
            self.optimizer.step()
        return loss.item()

    def get_action(self, obs):
        with torch.no_grad():
            dist_d, dist_c, val = self.net(torch.FloatTensor(obs).unsqueeze(0))
            return dist_d.sample().item(), dist_c.sample().item(), \
                   (dist_d.log_prob(dist_d.sample()) + dist_c.log_prob(dist_c.sample())).item(), val.item()

# ==========================================
# 4. 执行训练
# ==========================================
history_steps = []
history_avg_transfer = []
max_steps = -1
best_ep_log = {"anger": [], "charm": [], "wallet": []}

env = RomanceEnv()
agent = PPOPartnerAgent()

for ep in range(300):
    obs, _ = env.reset()
    done = False
    roll = {'states':[], 'tactical_acts':[], 'transfer_acts':[], 'log_probs':[], 'rewards':[], 'wallets':[], 'returns':[]}

    curr_ep_anger = []
    curr_ep_charm = []
    curr_ep_wallet = []
    while not done:
    # 记录每一步的状态
        curr_ep_anger.append(env.anger)
        curr_ep_charm.append(env.charm)
        curr_ep_wallet.append(env.wallet)
        roll['states'].append(obs)
        roll['wallets'].append(obs[2])
        a_d, a_c, lp, v = agent.get_action(obs)
        obs, r, done, _, info = env.step({"tactical": a_d, "transfer": np.array([a_c])})
        roll['tactical_acts'].append(a_d)
        roll['transfer_acts'].append(a_c)
        roll['log_probs'].append(lp)
        roll['rewards'].append(r)

    # 计算 Return
    G = 0
    returns = []
    for r in reversed(roll['rewards']):
        G = r + agent.gamma * G
        returns.insert(0, G)
    roll['returns'] = returns

    l = agent.update(roll)
    # 记录训练数据
    steps_count = len(roll['rewards'])
    avg_t = np.mean(roll['transfer_acts'])
    history_steps.append(steps_count)
    history_avg_transfer.append(avg_t)

    # 检查并更新最长回合的数据
    if steps_count > max_steps:
        max_steps = steps_count
        best_ep_log["anger"] = curr_ep_anger
        best_ep_log["charm"] = curr_ep_charm
        best_ep_log["wallet"] = curr_ep_wallet
    if ep % 10 == 0:
        avg_transfer = np.mean(roll['transfer_acts'])
        print(f"Epoch {ep} | Steps: {len(roll['rewards'])} | Avg Transfer: {avg_transfer:.1f} | Result: {info['reason']}")
```